

AI Ops Assignment 3

Madhav Arun Krishna – DA24B012

October 2026

1 Setup and Pipeline

- The experiment used approximately 2 GB of NYC Yellow Taxi parquet data from January 2024 through August 2026, together with the taxi zone lookup CSV.
- The machine had an AMD Ryzen 7 8845HS with 8 cores and 16 threads, 32 GB RAM, Linux 7.0.0-31, Python 3.12.3, Java 17, Spark 4.2.0 and Ray 2.58.0.
- Both clusters ran on this single laptop. Spark used one master and two worker processes with one core and 8 GB assigned to each worker. Ray used one local head node, two CPU slots and a 12 GiB object store.
- Spark and Ray were run separately under the same machine conditions.

Both programs perform the same preprocessing. They select the same eleven input columns, remove rows containing nulls, remove duplicates, format timestamps, and filter trips with non-positive duration or distance. The taxi zone lookup is joined once for pickup and once for drop-off. A custom Python function calculates average speed in miles per hour, and pickup hour is also added. The final data is written to parquet.

Spark uses a UTC SQL session because the taxi timestamps are timezone-naive. This prevents the laptop's Asia/Kolkata timezone from shifting the stored Spark timestamps. The independent validator compares the outputs without depending on their row order.

Framework	Total time (s)	UDF time (s)	Peak CPU (%)	Peak memory (MB)
Spark	660.783	392.830	261.5	12645.1
Ray	686.144	316.777	201.0	27039.3

Table 1: Measured benchmark results.

Both pipelines read 119,595,677 rows and wrote 94,001,243 rows. The validation result had matching columns, row counts and row hashes.

The first validation run had equal columns and row counts but an invalid row-hash parity match. The cause was that Spark used the laptop's Asia/Kolkata session timezone and shifted the timezone-naive taxi timestamps by 5 hours 30 minutes, while Ray kept them unchanged. Setting the Spark SQL session timezone to UTC fixed the timestamp representation, after which the complete validation passed.

2 Performance Comparison

Total Execution Time

Spark  660.783 s
Ray  686.144 s

Spark completed the full pipeline 25.361 seconds faster, which is about 3.7% of Ray's total time. Spark therefore won the overall run, although the two results were close.

Python UDF Time

Spark  392.830 s
Ray  316.777 s

Ray completed the custom Python transformation 76.053 seconds faster. This is about a 19.4% reduction from Spark's UDF time. Ray's Python-native actor execution avoided some of the boundary overhead involved when Spark executes a Python UDF outside its JVM execution engine.

Observed Peak CPU

Spark  261.5%
Ray  201.0%

The CPU value is the sum of the visible main processing workers in the captured `top` screenshot. Values above 100% are expected because multiple cores are being used. Spark showed a larger observed processing spike.

Observed Peak Memory

Spark  12,645.1 MB
Ray  27,039.3 MB

The memory value is the whole-system used-memory reading from `top`, converted from MiB to MB. It is not framework-only memory, but both values were collected in the same manner. Ray used much more memory and also caused swap use during the observed peak. Its object store and shuffle materialisation made memory the main weakness of this run.

3 Cluster and Correctness Evidence

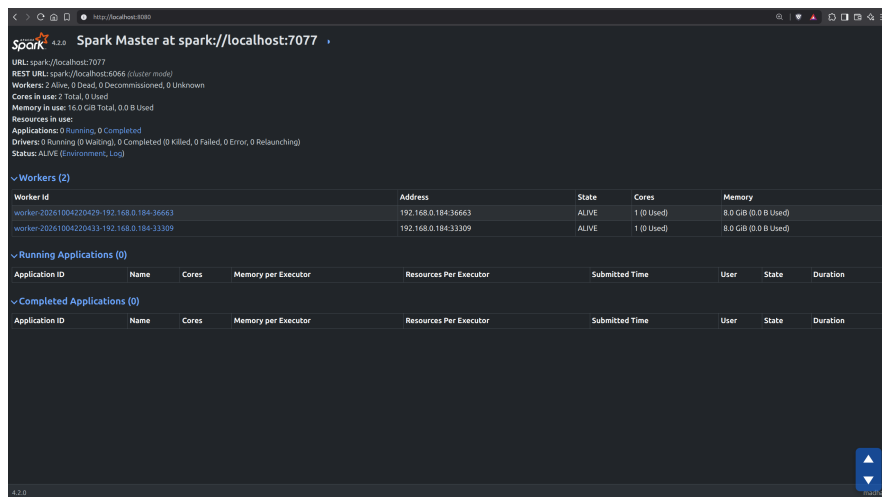


Figure 1: Spark Master UI showing two alive workers, one core and 8 GiB each.

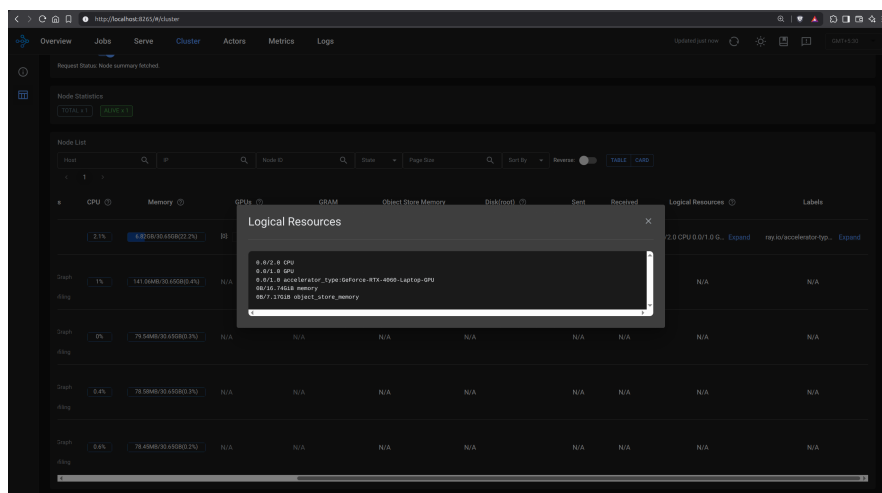


Figure 2: Ray Dashboard showing two logical CPU slots on the local Ray node.

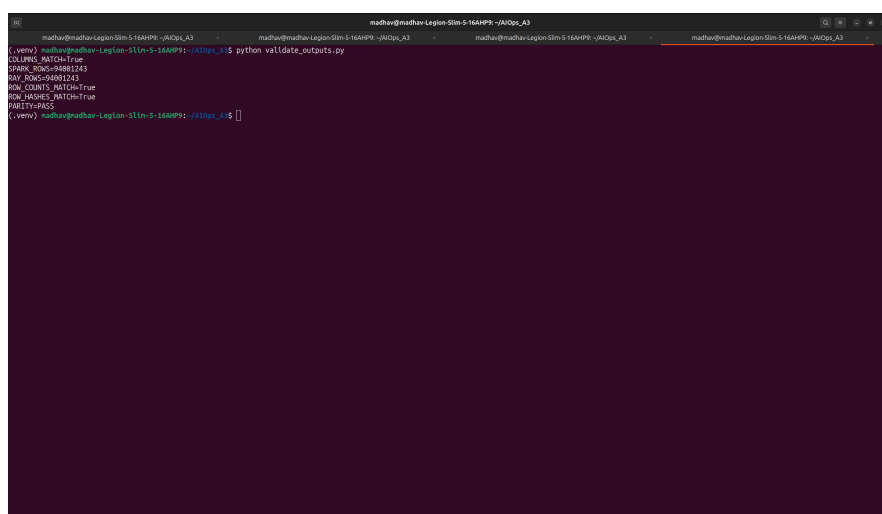


Figure 3: Bounded-memory validation showing equal row counts, matching row hashes and parity pass.

4 Discussion and Conclusion

Spark

- Spark was faster for the complete structured ETL workload and used considerably less observed system memory.
- Its DataFrame operations, joins and shuffle execution are a natural fit for large tabular pipelines.
- The Python UDF was the slowest measured section. The UDF crosses between Spark's JVM-based engine and Python workers, which reduces the advantage of Spark's native SQL execution.

Ray

- Ray was faster in the isolated Python UDF stage, which supports its Python-native advantage for AI-oriented transformations.
- The full pipeline was slower because duplicate removal and the two joins required expensive distributed shuffles and materialisation.
- Ray's memory usage was the major cost. The object store helped execution continue, but this workload placed much more pressure on the 32 GB laptop.

Performance Tuning Note

An LLM identified that Ray's global duplicate-removal group-by was producing oversized and memory-intensive shuffle work. Ray 2.58's hash-shuffle v2 was enabled with 64 output partitions and 256 MiB input batches. The object store was increased to 12 GiB to reduce backpressure and reactive spilling. Ray output was changed from append to overwrite for repeatable runs, and the batch UDF was given 50,000-row batches, two actor workers, and a list comprehension instead of DataFrame.apply with axis=1. The cleaning rules, joins, feature formula and output columns were not changed.

Winner

For this particular pipeline, **Spark is the winner**. It completed the complete job faster and used less observed memory. I would choose Spark for a BI-first project involving structured tables, SQL-style joins and aggregations. I would choose Ray for an AI-first project where the important work is Python-native model inference or custom functions, because it gives more flexible Python task and actor execution and had the faster UDF stage here.

The main limitation is that both clusters were formed from processes on one laptop rather than separate physical machines. Therefore, this compares the frameworks and their local distributed execution, but does not measure network transfer between machines. Even with that limitation, the same input, worker count, logic and output validation make the comparison fair for the available hardware.